

Exercise 2 Solutions

1. A mutex over the entire table is undesirable since it would unnecessarily restrict concurrency. Such a design would only permit a single insert, lookup or delete operation to be outstanding at any given time, even if they are to different hash bins. A mutex over each element in the doubly linked list would permit the greatest concurrency, but a correct, deadlock-free implementation has to ensure that all elements involved in a delete or insert operation, namely, up to three elements for a delete, or two elements and the hash bin for inserts/some deletes, are acquired in a well-defined order. A mutex over each hash bin is a compromise between these two solutions – it permits more concurrency than solution 1, and is easier to implement correctly than solution 2.

2.

a.

int N=50

int ApplicationForms = N; // Total number of applications present

b.

semaphore Line = 1; // Indicates the turn of the student in the line

semaphore Filled = 0; // Indicates that the 50 applications have been placed

semaphore FillApp = 0; // Notifies the admission officer to place 50 applications

c.

Applicant()

```
{
    wait(Line);
    if(ApplicationForms==0)
    {
        signal(FillApp);
        wait(Filled);
    }
    ApplicationForms--;
    signal(Line);
}
```

Officer()

```
while(True)
{
    wait(FillApp);
    ApplicationForms = N;
    signal(Filled);
}
```

3.

Smokers can enter the lounge at any time. If smokers in the lounge want to smoke, they must make sure that non-smokers are absent. If there are some non-smokers who are waiting to enter the lounge, the smokers who are not smoking cannot smoke any more. Non-smokers cannot enter the lounge if some smokers are smoking in the lounge.

a. Variable:

int smokingCount=nonSmoCount=0;//the number of smokers and nonsmokers in the lounge

b. Semaphore:

Semaphore smoking=mutexSmoker=mutexNonSmoker=enter=1;

c.

nonSmoker:

```
enterlounge(false)
{
    wait(enter);
    wait(mutexNonSmoker);
    if(nonSmoCount==0)
        wait(smoking);
    nonSmoCount++;
    signal(mutexNonSmoker);
    signal(enter);
}
//chat()
leavelounge(false)
{
    wait(mutexNonSmoker);
    nonSmoCount--;
    if(nonSmoCount==0)
        signal(smoking);
    signal(mutexNonSmoker);
}
```

Smoker:

```
enterlounge(true);
{
    wait(enter);
    signal(enter);
}
//chat()
smoke()
{
    wait(mutexSmoker)
    if(smokingCount==0)
        wait(smoking);
    smokingCount++;
    signal(mutexSmoker);
    //smoke
    wait(mutexSmoker);
    smokingCount--;
    If(smokingCount==0)
        signal(smoking);
    signal(mutexSmoker);
}
leavelounge(true) {};
```

4.

a) int searchercnt = 0;

b) semaphore mutex = searcher = inserter = deleter = 1;

c)

// Searcher

do{

 wait(mutex);

 wait(mutexcnt);

 wait(deleter);

 searchercnt ++;

 if(searchercnt == 1)

 wait(searcher);

 signal(deleter);

 signal(mutexcnt);

 signal(mutex)

 // searching

 wait(mutexcnt);

 searchercnt --;

 if(searchercnt == 0)

 signal(searcher);

 signal(mutexcnt);

} while (TRUE);

// Inserter

do{

 wait(mutex);

 wait(deleter);

 wait(inserter);

 signal(deleter);

 signal(mutex);

 // inserting

 signal(inserter);

} while (TRUE);

// Deleter

do{

 wait(mutex);

 wait(searcher);

 wait(inserter);

 wait(deleter);

 signal(mutex);

 // inserting

 signal(deleter);

 signal(inserter);

 signal(searcher);

} while (TRUE);